

Project Executable Files

Date	29 September 2026 – 2 October 2026
Team ID	6ab4fddea238dc999eb665e9
Project Name	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Maximum Marks	3 Marks

Project Executable Files

This section provides the executable source files, setup information, deployment/access details, repository and demonstration resources for PocketSmart AI. The evaluator can use the submitted source structure and setup instructions to run the FastAPI application locally or access the hosted deployment.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

PocketSmart AI project structure:

```
PocketSmart/  
├── main.py  
├── gemini_utils.py  
├── database.py  
├── schemas.py  
├── auth.py  
├── config.py  
├── requirements.txt  
├── .env.example  
├── .gitignore  
├── README.md  
├── templates/  
│   ├── base.html  
│   ├── index.html  
│   ├── login.html  
│   ├── register.html  
│   ├── dashboard.html  
│   ├── home_planner.html  
│   ├── party_planner.html  
│   ├── jewelry_planner.html  
│   ├── history.html  
│   ├── recommendation_details.html  
│   └── recommendation_result.html  
├── static/  
│   ├── css/styles.css  
│   ├── js/app.js  
│   └── uploads/  
└── tests/
```

Project Structure Notes

- `main.py` — FastAPI application entry point and route registration.
- `gemini_utils.py` — Gemini AI request/response handling and recommendation generation utilities.
- `database.py` — SQLite/database persistence and data-access logic.
- `schemas.py` — Pydantic request/response validation models.
- `auth.py` — authentication, password handling and protected-access logic.
- `config.py` — application configuration and environment-variable settings.
- `templates/` — Jinja2 HTML pages for authentication, dashboard and the three planners.
- `static/` — CSS, JavaScript and uploaded-image storage used by the web interface.
- `tests/` — project test cases and test support files.

Step 3: Deployment / Access Details

Field	Details
Deployed URL	https://pocket-smart-ai-complete.vercel.app/
Credentials (Demo)	Use the Register page to create an evaluator account. A verified hosted account could not be created from this document-generated environment, so no fabricated credentials are provided.
Hosting Provider	Vercel
Repository Link	https://github.com/Srinivasan-ramalingam/PocketSmart_AI_complete
Demo Link	https://www.loom.com/share/b9ce320e4df046db8dc89ec8aec84999

Access Notes

- The hosted URL is supplied by the project team and is intended for evaluator access.
- For the deployed application, use the Register page to create an account before accessing protected planner/history functionality.
- The repository contains the source code, project configuration and supporting files.
- The Loom link provides the project walkthrough/demo.

Step 4: Run Instructions

1. Install Python 3.x on the system and open a terminal in the PocketSmart project folder.
2. Create and activate a virtual environment, for example: `python -m venv .venv`
3. Activate the environment on Windows: `.venv\Scripts\activate`
4. Install project dependencies: `pip install -r requirements.txt`
5. Create a local environment file by copying `.env.example` to `.env`.
6. Add the required Gemini API key/configuration values to `.env` when live Gemini AI recommendations are required. Do not commit `.env` to GitHub.
7. Start the FastAPI application using Uvicorn: `uvicorn main:app --reload`
8. Open the local application URL shown by Uvicorn in a web browser.
9. Register an account, log in, and test the Home Interior, Party and Jewelry planners.
10. For Jewelry multimodal testing, use the supported image-upload flow and a valid image file.
11. Run the project tests from the project root with: `python -m pytest -q`

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Description	Workaround / Status
1	Gemini API access	Live AI generation requires the configured Gemini API credentials.	Set GEMINI_API_KEY / required environment configuration in <code>.env</code> .
2	External platform integrations	The project uses <code>safe/demo/search</code> links rather than claiming unsupported live third-party commerce APIs.	Use the provided recommendation/search links or integrate official APIs separately.
3	Hosted account access	A pre-created evaluator account was not available from the supplied project information.	Create an account through the deployed Register page before evaluation.